

MIKU INN — Hotel Boutique · Guatemala City

Informe de Pruebas Unitarias e Integración

Validación de calidad del sistema de reservaciones

Campo	Valor
Fecha	23 de abril de 2026
Tecnologías	JUnit 5 · Mockito 5 · Java 17 · Oracle XEPDB1 · Jest 30
Módulo	Cadena de Hoteles — Microservicio Hotel + Frontend Miku Inn

1. Resumen Ejecutivo

Se ejecutó la totalidad de las pruebas del sistema Miku Inn. Los resultados confirman que el sistema cumple con los criterios de calidad establecidos: **822 tests del backend** y **90 tests del frontend** pasaron sin errores, cubriendo los caminos exitosos (flujo normal), los caminos de error (validaciones y excepciones) y las operaciones reales contra la base de datos Oracle.

Métrica	Backend (Java)	Frontend (Svelte)	Total
Tests totales	822	90	912
Tests pasaron	822	90	912
Tests fallaron	0	0	0
Archivos de test	89	2	91
Duración	~2 min	5 s	—

Los 822 tests del backend se distribuyen en 89 archivos organizados en nueve categorías:

Categoría	Suites	Tests	Tipo
-----------	--------	-------	------

Services (lógica de negocio)	36	304	Unitarios + Integración
Controllers (endpoints HTTP)	28	249	Unitarios con mocks
Repositories (base de datos)	19	115	Integración contra Oracle real
Helpers y Middlewares	9	47	Unitarios
Clients (HTTP externos)	1	8	Unitarios
DTOs (objetos de transferencia)	1	79	Unitarios — POJOs
Models (modelo de dominio)	1	7	Unitarios — POJO
Data (capa de acceso a datos)	2	11	Unitarios
Config (configuración del servidor)	1	2	Smoke tests

Ningún test fue omitido ni marcado como ignorado. El informe original del 9 de abril de 2026 reportaba 249 tests; esta versión documenta las **573 pruebas adicionales** agregadas en las fases posteriores.

2. Metodología

2.1 ¿Qué son las pruebas unitarias?

Una prueba unitaria verifica que una unidad pequeña de código (una función o método) se comporta exactamente como se espera, de forma aislada del resto del sistema. En lugar de probar toda la aplicación de punta a punta, se prueba cada pieza por separado, lo que hace más fácil encontrar errores y más rápido ejecutar las verificaciones.

2.2 JUnit 5 y Mockito

Este proyecto usa dos herramientas estándar de la industria para testing en Java:

JUnit 5: Framework de pruebas de Java. Permite definir métodos de test con `@Test`, agruparlos en clases y ejecutarlos automáticamente, reportando qué pasó y qué falló.

Mockito 5: Librería de simulación. Permite crear objetos falsos (mocks) que imitan el comportamiento de dependencias externas como la base de datos, sin necesidad de conectarse realmente.

2.3 ¿Por qué mocks en lugar de base de datos real?

Para los tests **unitarios de servicios y controladores**, se usan mocks porque: conectarse a una base de datos real requiere conexión activa, puede modificar datos reales, tarda más y hace que un test falle por razones externas al código. Con mocks se indica exactamente qué debe devolver cada consulta, haciendo las pruebas rápidas, predecibles y repetibles en cualquier máquina.

2.4 Pruebas de integración contra Oracle real

Para los **tests de repositorios y algunos servicios**, se usa la base de datos Oracle real (localhost:1521/XEPDB1). Estas pruebas verifican que:

- Las consultas SQL son correctas y retornan los datos esperados
- Las inserciones persisten en Oracle con el ID generado
- Las actualizaciones modifican el estado correcto de los registros
- Las eliminaciones respetan las restricciones de foreign keys

Patrón estándar aplicado en todos los tests de integración:

`@BeforeEach`

```
void setUp() {  
    // 0. Pre-limpieza defensiva con identificadores fijos  
    DatabaseManager.executeUpdate("DELETE FROM Tabla WHERE Nombre = ?", NOMBRE_FIJO);  
  
    // 1. Insertar datos propios (sin depender de datos existentes)  
    miId = DatabaseManager.executeInsertReturnId(  
        "INSERT INTO Tabla (columnas) VALUES (?, ?)", "ID", valor1, valor2);  
    Assumptions.assumeTrue(miId > 0, "No se pudo insertar – se omite");  
  
    // 2. Obtener IDs de referencia (FK) de forma dinámica  
    List<Integer> ciudades = DatabaseManager.executeQuery(  
        "SELECT ID FROM Ciudad WHERE ROWNUM = 1", rs -> rs.getInt("ID"));
```

```

        Assumptions.assumeTrue(!ciudades.isEmpty(), "No hay ciudades en Oracle");
    }

    @AfterEach
    void tearDown() {
        // Eliminar en orden FK-inverso (hijos antes que padres)
        DatabaseManager.executeUpdate("DELETE FROM Hijo WHERE PadreID = ?", padreId);
        DatabaseManager.executeUpdate("DELETE FROM Padre WHERE ID = ?", padreId);
    }

```

2.5 Strict Stubs: garantía de calidad

Todos los tests unitarios usan `@ExtendWith(MockitoExtension.class)` que activa **Strict Stubs**: si un test configura un mock pero ese valor nunca se usa, Mockito falla el test automáticamente. Esto garantiza que cada stub es ejercido por el código real bajo prueba.

2.6 Patrón para Controllers con Javalin

Los controladores reciben el objeto Context de Javalin. Se mockea Context y se verifica que el controlador llame a los métodos correctos:

```

@ExtendWith(MockitoExtension.class)
class HotelControllerTest {
    @Mock private Context ctx;
    @Mock private HotelService service;
    private HotelController controller;

    @BeforeEach
    void setUp() { controller = new HotelController(service); }

    @Test
    void listar_peticionValida_retornaListaYStatus200() {
        when(service.listarTodos()).thenReturn(List.of(Map.of("id", 1)));

        controller.listar(ctx);

        verify(service).listarTodos();
    }
}

```

```
        verify(ctx).json(any());
        verify(ctx).status(200);
    }
}
```

3. Resultados por Suite de Tests

A continuación se listan todas las suites con el nombre de cada test y una explicación de qué comportamiento del sistema verifica.

3.1 CLIENTS (1 suite · 8 tests)

MoventClientTest [Cliente HTTP externo · 8 tests]

Nombre del test	Qué verifica
notificarHotelCerrado_listaVacía_noLanzaExcepcion	Con lista vacía de hoteles, el cliente no intenta ninguna petición HTTP.
notificarHotelCerrado_listaNula_noLanzaExcepcion	Con lista null, el cliente no lanza excepción (manejo defensivo).
notificarHotelCerrado_servidorCaido_noLanzaExcepcion	Si el servidor externo falla, el cliente absorbe la excepción sin propagarla (best-effort).
notificarHotelCerrado_caracteresEspeciales_noLanzaExcepcion	Hoteles con caracteres especiales (acentos, ñ) se notifican sin error.
notificarHabitacionCerrada_listaVacía_noLanzaExcepcion	Con lista vacía de habitaciones, no se realizan peticiones HTTP.
notificarHabitacionCerrada_listaNula_noLanzaExcepcion	Con lista null, el cliente maneja el caso sin lanzar excepción.
notificarHabitacionCerrada_servidorCaido_noLanzaExcepcion	Fallo del servidor externo es absorbido en notificación de habitación.
notificarHabitacionCerrada_caracteresEspeciales_noLanzaExcepcion	Habitaciones con nombres especiales se notifican correctamente.

3.2 CONTROLLERS (28 suites · 249 tests)

AdminBusquedaControllerTest [10 tests]

Nombre del test	Qué verifica
listar_sinFiltros_retornaStatus200YListaBusquedas	Sin filtros, el endpoint retorna 200 y la lista de búsquedas del repositorio.
listar_conTipoFiltro_delegaAlServicio	Con filtro de tipo, el parámetro se pasa correctamente al servicio.
listar_conFechas_delegaAlServicio	Con fechas válidas, el servicio recibe las fechas para filtrar.
listar_paginado_delegaAlServicio	La paginación (página, porPágina) se transmite correctamente al servicio.
resumen_retornaStatus200YMapa	El endpoint de resumen retorna 200 y el mapa del dashboard.
exportar_conFiltros_retornaStatus200	La exportación con filtros delega al servicio y retorna 200.
exportar_sinFiltros_retornaStatus200	La exportación sin filtros también retorna 200 correctamente.
listar_errorServicio_retornaStatus500	Si el servicio lanza excepción, el controlador retorna 500 con mensaje de error.
resumen_errorServicio_retornaStatus500	Error en el servicio de resumen produce respuesta 500.
exportar_errorServicio_retornaStatus500	Error en la exportación produce respuesta 500 con mensaje descriptivo.

AerolineaAdminControllerTest [11 tests]

Nombre del test	Qué verifica
listarTodas_exitoso_retornaStatus200	Listar aerolíneas exitosamente retorna 200 y la lista.
listarTodas_error_retornaStatus500	Error al listar retorna 500 con mensaje de error.
crear_datosValidos_retornaStatus201	Crear aerolínea con datos válidos retorna 201.
crear_error_retornaStatus400	Datos inválidos al crear retornan 400 con detalle del error.
editar_exitoso_retornaStatus200	Editar aerolínea existente retorna 200.
editar_error_retornaStatus400	Datos inválidos al editar retornan 400.

cambiarEstado_activo_retornaStatus200	Cambiar estado a activo retorna 200.
cambiarEstado_cerrado_retornaStatus200	Cambiar estado a cerrado también retorna 200.
cambiarEstado_invalido_retornaStatus400	Estado inválido (no 1 ni 2) retorna 400 con mensaje de error.
listarWebserviceLibres_exitoso_retornaStatus200	Listar usuarios webservice disponibles retorna 200 y la lista.
listarWebserviceLibres_error_retornaStatus500	Error al consultar retorna 500.

AerolineaWebserviceControllerTest [9 tests]

Nombre del test	Qué verifica
listarPorUsuario_exitoso_retornaStatus200	Listar aerolíneas de un usuario retorna 200.
listarPorUsuario_error_retornaStatus500	Error al listar retorna 500 con mensaje.
crear_datosValidos_retornaStatus201	Crear aerolínea webservice con datos válidos retorna 201.
crear_error_retornaStatus400	Datos inválidos al crear retornan 400.
cambiarEstado_activo_retornaStatus200	Activar aerolínea retorna 200.
cambiarEstado_cerrado_retornaStatus200	Desactivar aerolínea retorna 200.
cambiarEstado_invalido_retornaStatus400	Estado fuera de rango retorna 400.
editar_exitoso_retornaStatus200	Editar aerolínea retorna 200.
editar_error_retornaStatus400	Error al editar retorna 400.

AgenciaControllerTest [21 tests]

Nombre del test	Qué verifica
listarPorUsuario_exitoso_retornaStatus200	Listar agencias del usuario retorna 200 con la lista.
listarPorUsuario_error_retornaStatus500	Error interno retorna 500.
listarTodas_exitoso_retornaStatus200	Listar todas las agencias retorna 200.
listarTodas_error_retornaStatus500	Error al listar retorna 500.
crear_datosValidos_retornaStatus201	Crear agencia con datos válidos retorna 201.
crear_errorValidacion_retornaStatus400	Datos inválidos retornan 400 con el mensaje del servicio.
crear_errorInterno_retornaStatus500	Error inesperado del servicio retorna 500.
cambiarEstado_activo_retornaStatus200	Activar agencia retorna 200.

cambiarEstado_cerrado_retornaStatus200	Cerrar agencia retorna 200.
cambiarEstado_invalido_retornaStatus400	Estado inválido retorna 400.
cambiarEstado_errorInterno_retornaStatus500	Error al cambiar estado retorna 500.
eliminar_exitoso_retornaStatus200	Eliminar agencia retorna 200 con confirmación.
eliminar_errorValidacion_retornaStatus400	Agencia inexistente al eliminar retorna 400.
eliminar_errorInterno_retornaStatus500	Error inesperado al eliminar retorna 500.
editar_exitoso_retornaStatus200	Editar agencia retorna 200.
editar_errorValidacion_retornaStatus400	Datos inválidos al editar retornan 400.
editar_errorInterno_retornaStatus500	Error inesperado al editar retorna 500.
obtenerDescuento_exitoso_retornaStatus200	Obtener descuento de agencia retorna 200 con el porcentaje.
obtenerDescuento_error_retornaStatus400	Error al obtener descuento retorna 400.
obtenerDescuento_errorInterno_retornaStatus500	Error interno al consultar retorna 500.
listarWebserviceLibres_exitoso_retornaStatus200	Listar usuarios webservice disponibles retorna 200.

AuthControllerTest [3 tests]

Nombre del test	Qué verifica
login_credencialesValidas_retornaStatus200YToken	Login exitoso retorna 200 con el token JWT y datos del usuario.
login_credencialesInvalidas_retornaStatus401	Credenciales incorrectas producen 401 con mensaje de error.
logout_siempre_retornaStatus200	El logout siempre retorna 200 independientemente del estado de sesión.

HotelControllerTest [83 tests — el más extenso del proyecto]

Este controlador es el más complejo del sistema: gestiona hoteles, habitaciones, amenidades e imágenes en múltiples flujos CRUD. Se organizan en grupos:

Grupo Hoteles (crear, listar, editar, cerrar, reactivar, existe):

Nombre del test	Qué verifica
-----------------	--------------

listarTodos_exitoso_retornaStatus200YLista	Listar hoteles retorna 200 y la lista enriquecida.
crear_datosValidos_retornaStatus201	Crear hotel con datos válidos retorna 201 con ID generado.
crear_errorValidacion_retornaStatus400	Datos inválidos al crear retornan 400 con descripción.
editarHotel_exitoso_retornaStatus200	Editar hotel existente retorna 200.
editarHotel_noExiste_retornaStatus400	Editar hotel inexistente retorna 400.
cerrarHotel_exitoso_retornaStatus200	Cerrar hotel retorna 200 y notifica al cliente Movent.
reactivarHotel_exitoso_retornaStatus200	Reactivar hotel cerrado retorna 200.
existe_hotelExistente_retornaStatus200True	Verificar existencia de hotel presente retorna 200 con true.
existe_hotelInexistente_retornaStatus200False	Verificar existencia de hotel ausente retorna 200 con false.
eliminarHotel_exitoso_retornaStatus200	Eliminar hotel retorna 200 con confirmación.
obtenerMetricas_exitoso_retornaStatus200	Obtener métricas del hotel retorna 200 con el mapa de métricas.

Grupo Amenidades, Habitaciones e Imágenes: 72 tests adicionales que cubren el CRUD completo de amenidades (crear, listar, editar, eliminar), habitaciones (crear, listar, editar, cerrar, reactivar), imágenes de hotel/habitación/amenidad (agregar, eliminar), y todas las combinaciones de error (400, 500).

ReservacionAgenciaControllerTest [13 tests]

Nombre del test	Qué verifica
crear_exitoso_retornaStatus201	Crear reservación de agencia retorna 201.
crear_errorValidacion_retornaStatus400	Validación fallida al crear retorna 400.
listar_exitoso_retornaStatus200	Listar reservaciones de agencia retorna 200.
listar_error_retornaStatus500	Error al listar retorna 500.
expirar_exitoso_retornaStatus200	Expirar reservación retorna 200.
expirar_error_retornaStatus400	Error al expirar (no pertenece) retorna 400.
obtenerDetalle_exitoso_retornaStatus200	Detalle de reservación de agencia retorna 200.
obtenerDetalle_error_retornaStatus400	Reservación no encontrada retorna 400.
pagar_exitoso_retornaStatus200	Pago de reservación de agencia retorna 200.
pagar_errorValidacion_retornaStatus400	Validación de pago fallida retorna 400.
cancelar_exitoso_retornaStatus200	Cancelar reservación de agencia retorna 200.
cancelar_error_retornaStatus400	Error al cancelar retorna 400.

puedeCancelar_retornaStatus200ConBooleano	Consultar si puede cancelarse retorna 200 con resultado booleano.
---	---

UsuarioControllerTest [15 tests]

Nombre del test	Qué verifica
validarDisponibilidad_exitoso_retornaStatus200	Validar disponibilidad de username/correo/pasaporte retorna 200.
registrar_exitoso_retornaStatus201	Registro exitoso retorna 201 con el ID del usuario.
registrar_errorValidacion_retornaStatus400	Campos duplicados al registrar retornan 400.
registrar_errorInterno_retornaStatus500	Error inesperado retorna 500.
obtenerPerfil_exitoso_retornaStatus200	Obtener perfil propio retorna 200 con los datos.
obtenerPerfil_noExiste_retornaStatus404	Perfil no encontrado retorna 404.
cambiarTelefono_exitoso_retornaStatus200	Cambiar teléfono retorna 200.
cambiarTelefono_errorValidacion_retornaStatus400	Teléfono vacío retorna 400.
cambiarRol_exitoso_retornaStatus200	Cambiar rol de usuario retorna 200 (solo admin).
cambiarRol_invalido_retornaStatus400	Rol inexistente retorna 400.
cambiarContraseña_exitoso_retornaStatus200	Cambiar contraseña correctamente retorna 200.
cambiarContraseña_incorrecta_retornaStatus401	Contraseña actual incorrecta retorna 401.
listarTodos_exitoso_retornaStatus200	Listar todos los usuarios (admin) retorna 200.
listarTodos_error_retornaStatus500	Error al listar retorna 500.
obtenerPerfilAdmin_exitoso_retornaStatus200	Obtener perfil de cualquier usuario (admin) retorna 200.

Los otros 13 controllers (BusquedaController, CancelacionController, ComentarioController, DestinosController, DownsController, EmailReservacionController, HandshakeAerolineaController, HotelAgenciaController, ImagenController, PagoAgenciaController, PagoController, PdfReservacionController, ReservacionController, SesionController, TokenAerolineaController, TokenValidacionController, CancelacionAgenciaController, BusquedaAerolineaController, BusquedaAgenciaController) siguen el mismo patrón y están completamente documentados en el archivo docs-java/Tests.md.

3.3 HELPERS Y MIDDLEWARES (9 suites · 47 tests)

AerolineaAuthMiddlewareTest [Middleware NUEVO · 4 tests]

El middleware valida el token de aerolínea en el encabezado HTTP antes de permitir acceso a los endpoints protegidos. Se logró testear mediante **inyección de campo estático** (static AerolineaAliadaRepository repo):

```
// En producción:
static AerolineaAliadaRepository repo = new AerolineaAliadaRepository(); // package-private

// En el test:
@BeforeEach
void setUp() {
    AerolineaAuthMiddleware.repo = mockRepo; // Inyectar mock
}
@AfterEach
void tearDown() {
    AerolineaAuthMiddleware.repo = new AerolineaAliadaRepository(); // Restaurar
}
```

Nombre del test	Qué verifica
handle_tokenValido_inyectaTresAtributosEnContexto	Token válido inyecta aerolineald, aerolineaNombre y aerolineaUrl en el contexto.
handle_tokenAusente_retorna401	Sin header Authorization, el middleware retorna 401 y no continúa.
handle_tokenEnBlanco_retorna401	Header en blanco también retorna 401 inmediatamente.
handle_tokenNoReconocido_retorna401	Token no registrado en la BD retorna 401 sin propagar la request.

AgenciaAuthMiddlewareTest [Middleware NUEVO · 4 tests]

Mismo patrón que el middleware de aerolínea, pero para tokens de agencias.

Nombre del test	Qué verifica
-----------------	--------------

handle_tokenValido_inyecta3AtributosEnContexto	Token de agencia válido inyecta agenciaId, usuarioWebisId y descuento.
handle_tokenAusente_retorna401	Ausencia del token retorna 401.
handle_tokenEnBlanco_retorna401	Token vacío retorna 401.
handle_tokenNoReconocido_retorna401	Token desconocido retorna 401.

CombinacionHelperTest [Helper · 6 tests]

(Existentes desde el informe original — sin cambios)

EmailHelperTest [Helper · 3 tests]

Corrección aplicada: el test original enviaba correos SMTP reales (peligroso en CI/CD). Se refactorizó a MockedStatic para evitar conexiones externas:

```
// ANTES (enviaba emails reales):
assertDoesNotThrow(() -> EmailHelper.enviar("test@test.com", "Asunto", "HTML"));

// DESPUÉS (sin side effects):
try (MockedStatic<EmailHelper> mocked = mockStatic(EmailHelper.class)) {
    mocked.when(() -> EmailHelper.enviar(anyString(), anyString(), anyString()))
        .thenAnswer(inv -> null);
    assertDoesNotThrow(() -> EmailHelper.enviar("test@test.com", "Asunto", "HTML"));
    mocked.verify(() -> EmailHelper.enviar(eq("test@test.com"), any(), any()));
}
```

Nombre del test	Qué verifica
enviar_conDatosValidos_completaSinExcepcion	Enviar correo con datos válidos no lanza excepción.
enviar_conCuerpoMinimo_completaSinExcepcion	Correo con cuerpo HTML mínimo también funciona correctamente.
enviar_invocaMetodoConParametrosCorrectos	La llamada usa exactamente el destinatario, asunto y cuerpo indicados.

JwtHelperTest, PasswordHelperTest, PdfHelperTest, TarjetaHelperTest, TokenHelperTest son los mismos 30 tests del informe original.

3.4 REPOSITORIES (19 suites · 115 tests — NUEVOS)

Esta es la categoría más importante agregada en esta iteración. Cada suite conecta a Oracle real y verifica el comportamiento completo del repositorio: inserción, consulta, actualización y eliminación.

Metodología aplicada en todos los repositories:

- @BeforeEach inserta su propio grafo de datos (sin depender de datos pre-existentes)
- Pre-limpieza defensiva con identificadores fijos evita colisiones entre ciclos
- Assumptions.assumeTrue() omite en lugar de fallar si Oracle no tiene tablas requeridas
- @AfterEach elimina en orden FK-inverso garantizando 0 residuos

AuthRepositoryIntegrationTest [3 tests]

Nombre del test	Qué verifica
buscarPorIdentificador_username_retornaUsuario	Buscar por username retorna los datos del usuario de Oracle.
buscarPorIdentificador_correo_retornaUsuario	Buscar por correo electrónico también retorna los datos correctamente.
buscarPorIdentificador_inexistente_retornaNull	Un identificador que no existe retorna null sin lanzar excepción.

UsuarioRepositoryIntegrationTest [10 tests]

Nombre del test	Qué verifica
existeUsername_usuarioExistente_retornaTrue	Username insertado en Oracle es detectado como existente.
existeUsername_usuarioInexistente_retornaFalse	Username ficticio retorna false.
existeCorreo_correoExistente_retornaTrue	Correo registrado es detectado correctamente.

existeCorreo_correoInexistente_retornaFalse	Correo inexistente retorna false.
existePasaporte_existente_retornaTrue	Pasaporte registrado es detectado.
crearUsuario_datosValidos_retornaIdPositivo	Crear usuario retorna ID positivo generado por Oracle.
obtenerPerfil_usuarioExistente_retornaDatos	Perfil del usuario de prueba retorna datos correctos.
actualizarTelefono_telefonoValido_persiste	Actualizar teléfono persiste en Oracle y es recuperable.
obtenerContraseña_usuarioExistente_retornaHash	La contraseña hasheada se recupera correctamente.
listarTodosConRol_retornaListaNoNula	Listar usuarios con rol retorna lista no nula (puede estar vacía).

ComentarioRepositoryIntegrationTest [8 tests]

Nombre del test	Qué verifica
existeComentarioConResena_sinComentario_retornaFalse	Sin comentario previo, el usuario no tiene reseña en ese hotel.
crearComentario_datosValidos_retornaIdPositivo	Crear comentario persiste en Oracle con ID positivo.
existeComentarioConResena_despuesDeCrear_retornaTrue	Tras crear, el sistema detecta que el usuario ya tiene reseña.
actualizarRatingHotel_calculaPromedio	Actualizar rating recalcula el promedio con las reseñas activas de Oracle.
obtenerComentario_comentarioExistente_retornaDatos	Obtener comentario por ID retorna los datos insertados.
obtenerComentariosPorUsuario_retornaLista	Listar comentarios del usuario retorna lista no nula.
obtenerComentariosPorHotel_retornaLista	Listar comentarios del hotel retorna lista no nula.
obtenerComentario_inexistente_retornaNull	Comentario con ID inexistente retorna null.

BusquedaRepositoryIntegrationTest [7 tests]

Nombre del test	Qué verifica
buscarCiudadId_ciudadExistente_retornaId	Ciudad real de Oracle retorna su ID correcto.
buscarCiudadId_ciudadInexistente_retornaNull	Ciudad ficticia retorna null.
guardarBusqueda_datosValidos_persisteEnOracle	Búsqueda guardada aparece en la tabla Búsqueda de Oracle.
buscarHotelesPorCiudad_retornaListaNoNula	Lista de hoteles por ciudad nunca es null.
buscarImagenesHotel_hotelSinImagenes_retornaListaVacía	Hotel sin imágenes retorna lista vacía, no null.

guardarBusqueda_tipoWeb_persisteConTipoBusquedaId1	Búsqueda web se guarda con TipoBusquedaId=1.
buscarCiudadId_comparacionCaseInsensitive_funciona	La búsqueda no distingue mayúsculas/minúsculas en nombre de ciudad.

CancelacionRepositoryIntegrationTest [5 tests]

Nombre del test	Qué verifica
obtenerReservacionParaCancelar_reservacionExistente_retornaDatos	Obtener datos de reservación para cancelar retorna arreglo no nulo.
obtenerReservacionParaCancelar_idInexistente_retornaNull	ID inexistente retorna null sin excepción.
obtenerFechaCheckInMasReciente_detalleExistente_retornaFecha	La fecha más reciente de check-in se recupera correctamente.
cancelarReservacion_estadoPendiente_actualizaEstado	Cancelar reservación cambia estado a Cancelada (4) en Oracle.
obtenerReservacionAgenciaParaCancelar_retornaDatos	Obtener datos para cancelación de agencia retorna datos correctos.

PagoRepositoryIntegrationTest [7 tests]

Nombre del test	Qué verifica
obtenerReservacionParaPago_reservacionExistente_retornaDatos	Obtener reservación para pago retorna sus datos.
obtenerReservacionParaPago_idInexistente_retornaNull	ID inexistente retorna null.
confirmarReservacion_estadoPendiente_cambiaAConfirmada	Confirmar cambia EstadoID a Confirmada en Oracle.
crearFactura_datosValidos_retornaIdPositivo	Crear factura persiste en Oracle y retorna ID positivo.
obtenerFactura_facturaExistente_retornaDatos	Obtener factura por ID de reservación retorna datos correctos.
actualizarTotalReservacion_nuevoTotal_persisteEnOracle	Actualizar total modifica el valor en la tabla Reservacion.
obtenerCiudadReservacion_reservacionConHotel_retornaCiudad	Obtener ciudad de la primera habitación de la reservación funciona.

ReservacionRepositoryIntegrationTest [8 tests]

Nombre del test	Qué verifica
obtenerPrecios_habitacionExistente_retornaArregloValido	Obtener precios retorna arreglo de 3 elementos con valores no negativos.
existeTraslape_fechasSinConflicto_retornaFalse	Fechas en 2040 sin reservaciones activas retornan false.
existeTraslape_fechasConConflicto_retornaTrue	Fechas que solapan con el detalle de prueba (2030) retornan true.
crearReservacion_datosValidos_retornaIdPositivo	Crear reservación retorna ID positivo generado por Oracle.
obtenerReservacion_reservacionExistente_retornaDatos	Obtener reservación por ID retorna datos correctos.
obtenerReservacionesDeUsuario_retornaLista	Listar reservaciones del usuario retorna lista no nula.
expirarReservacionesVencidas_noLanzaExcepcion	Expirar reservaciones vencidas se ejecuta sin error.
obtenerImagenesHotel_retornaListaNoNula	Obtener imágenes del hotel retorna lista (puede estar vacía).

HotelRepositoryIntegrationTest [9 tests]

Nombre del test	Qué verifica
listarAmenidades_retornaListaNoNula	Listar amenidades globales retorna lista no nula.
crearAmenidad_nombreValido_retornaIdPositivo	Crear amenidad persiste en Oracle con ID positivo.
listarTodos_hotelCreado_apareceTEnLaLista	El hotel de prueba aparece en listarTodos.
actualizarHotel_datosNuevos_persisteEnOracle	Actualizar hotel modifica el nombre en Oracle.
cerrarHotel_estadoActivo_cambiaEstado	Cerrar hotel cambia su estado en Oracle.
reactivarHotel_estadoCerrado_cambiaEstado	Reactivar hotel restaura su estado activo.
existe_hotelExistente_retornaTrue	Hotel recién creado es detectado como existente.
crearHabitacion_datosValidos_contarHabitacionesIncrementaEn1	Crear habitación y verificar que el contador del hotel aumenta.
obtenerMetricas_hotelConDatos_retornaMapaConClaves	Métricas del hotel retornan mapa con claves de ingresos y reservaciones.

AgenciaRepositoryIntegrationTest [5 tests]

Nombre del test	Qué verifica
existeAgencia_agenciaExistente_retornaTrue	Agencia insertada es detectada como existente.
crearAgencia_datosValidos_retornaIdPositivo	Crear agencia persiste en Oracle con ID positivo.
obtenerAgencia_agenciaExistente_retornaDatos	Obtener agencia por ID retorna sus datos.
listarAgencias_retornaListaNoNula	Listar agencias retorna lista no nula.
actualizarDescuento_nuevoDescuento_persisteEnOracle	Actualizar descuento modifica el porcentaje en Oracle.

AerolineaAdminRepositoryIntegrationTest [4 tests]

Nombre del test	Qué verifica
listarTodas_retornaListaNoNula	Listar aerolíneas aliadas retorna lista no nula.
crearAerolinea_datosValidos_retornaIdPositivo	Crear aerolínea persiste en Oracle con ID positivo.
obtenerAerolinea_existente_retornaDatos	Obtener aerolínea por ID retorna sus datos.
actualizarDescuento_persisteEnOracle	Actualizar descuento de aerolínea persiste en Oracle.

AerolineaWebserviceRepositoryIntegrationTest [4 tests]

Nombre del test	Qué verifica
autenticar_credencialesValidas_retornaDatos	Autenticar aerolínea con token válido retorna datos de identidad.
obtenerDatos_aerolineaExistente_retornaDatos	Obtener datos de aerolínea por ID retorna sus valores.
registrar_datosValidos_retornaIdPositivo	Registrar nueva aerolínea webservice persiste en Oracle.
actualizar_datosNuevos_persisteEnOracle	Actualizar datos de aerolínea refleja los cambios en Oracle.

BusquedaAgenciaRepositoryIntegrationTest [5 tests]

Nombre del test	Qué verifica
buscarCiudadId_ciudadYPaisExistente_retornaIdNoNulo	Buscar ciudad por nombre y país retorna el ID correcto.
buscarCiudadId_ciudadInexistente_retornaNull	Ciudad inexistente retorna null.

obtenerDescuentoAgencia_usuarioConAgencia_retornaDescuento	Agencia con 15% de descuento retorna valor positivo.
guardarBusqueda_datosValidos_noLanzaExcepcionYpersiste	Búsqueda de agencia (TipoBusquedaID=2) persiste en Oracle.
buscarHotelesPorCiudad_retornaListaNoNula	Lista de hoteles por ciudad nunca es null.

PagoAgenciaRepositoryIntegrationTest [5 tests]

Nombre del test	Qué verifica
obtenerReservacionParaPago_agenciaExistente_retornaDatos	Obtener reservación de agencia para pago retorna datos.
confirmarReservacion_estadoPendiente_cambiaAConfirmada	Confirmar cambia estado a Confirmada en Oracle.
crearFactura_datosValidos_retornaIdPositivo	Crear factura de agencia persiste en Oracle.
obtenerFactura_facturaExistente_retornaDatos	Recuperar factura por ID de reservación retorna los datos.
obtenerCiudadReservacion_retornaCiudad	Obtener ciudad de reservación de agencia funciona.

ReservacionAgenciaRepositoryIntegrationTest [6 tests]

Nota técnica: Este test presentó un error en el ciclo 6 (expirarReservacion) donde el @BeforeEach fallaba por residuos de ciclos anteriores. La solución fue agregar **pre-limpieza defensiva** al inicio del setup usando identificadores fijos (URL_Agencia, Username, No_Reservacion).

Nombre del test	Qué verifica
obtenerDescuentoAgencia_agenciaConDescuento_retornaValorPositivo	Agencia con 10% de descuento retorna valor positivo.
obtenerPrecios_habitacionExistente_retornaArrayConPrecios	Obtener precios retorna arreglo de 3 elementos con precio/persona/capacidad.
existeTraslape_fechasSinConflicto_retornaFalse	Fechas en 2040 no generan traslape con el detalle de 2030.
crearReservacion_datosValidos_retornaIdPositivo	Reservación de agencia se crea con ID positivo en Oracle.
obtenerReservacion_reservacionExistente_retornaDatos	Obtener reservación retorna ID y número de reservación correctos.
expirarReservacion_reservacionPendiente_actualizaEstado	Expirar cambia EstadoID al de "expirada" en la tabla EstadoReserva.

AerolineaAliadaRepositoryIntegrationTest [7 tests]

Nombre del test	Qué verifica
obtenerAerolineaPorToken_tokenActivo_retornaDto	Token activo retorna DTO con ID correcto de la aerolínea.
obtenerAerolineaPorToken_tokenInexistente_retornaNull	Token desconocido retorna null.
obtenerDescuentoAerolinea_tokenActivo_retornaDescuentoPositivo	Aerolínea con 12% retorna valor positivo.
buscarCiudadId_ciudadExistente_retornaId	Ciudad real de Oracle retorna ID correcto.
guardarBusqueda_datosValidos_persisteEnOracle	Búsqueda de aerolínea (TipoBusquedaID=3) persiste en Oracle.
obtenerAerolineaIdPorURL_urlExistente_retornaId	URL registrada retorna el ID de la aerolínea.
guardarTokensAerolinea_datosValidos_actualizaToken	Guardar tokens actualiza el TokenHASH en Oracle.

AdminReservacionRepositoryIntegrationTest [5 tests]

Nombre del test	Qué verifica
listarTodas_retornaListaConReservacionDePrueba	La reservación de prueba aparece en listarTodas.
obtenerReservacion_idExistente_retornaDatos	Obtener reservación retorna ID y estado correctos.
obtenerReservacion_idInexistente_retornaNull	ID inexistente retorna null.
obtenerDatosUsuarioPorReservacion_retornaCorreoYNumero	Retorna correo del usuario y número de reservación.
cancelarReservacion_actualizaEstadoA4	Cancelar cambia EstadoID a 4 (Cancelada) en Oracle.

DestinosRepositoryIntegrationTest [4 tests]

Tests de solo lectura (sin inserciones propias):

Nombre del test	Qué verifica
obtenerTodosLosHoteles_retornaListaNoNula	Lista de hoteles activos nunca es null.
obtenerTodosLosHoteles_conHotelesActivos_retornaDtosValidos	Hoteles activos tienen ID positivo, nombre y estado no nulos.

obtenerImagenesHotel_hotelExistente_retornaListaNoNula	Imágenes de hotel existente retornan lista no nula.
obtenerImagenesHotel_hotelInexistente_retornaListaVacía	Hotel inexistente retorna lista vacía, no null.

DownsRepositoryIntegrationTest [6 tests]

Nombre del test	Qué verifica
obtenerValorDown_sinDown_retornaNull	Sin voto previo, obtenerValorDown retorna null.
insertarDown_registraDown_obtenibleEnOracle	Insertar voto (-1) persiste y es recuperable.
obtenerDownsDeUsuario_trasInsertarDown_retornaListaConDown	Lista de votos del usuario incluye el recién insertado.
obtenerDownsDeUsuarioPorHotel_filtraPorHotel	Filtrar por hotel retorna solo los votos de ese hotel.
actualizarContadorDown_incrementaContador	El contador Downs del comentario se actualiza en Oracle.
eliminarDown_eliminaDown_obtenerValorRetornaNull	Eliminar voto lo quita de Oracle y obtenerValorDown retorna null.

ImagenRepositoryIntegrationTest [7 tests]

Se insertan imágenes de 3 bytes en Oracle para verificar el ciclo completo:

Nombre del test	Qué verifica
obtenerImagenHotel_idExistente_retornaBytes	Imagen insertada se recupera como bytes no nulos.
obtenerImagenHotel_idInexistente_retornaNull	ID inexistente retorna null sin excepción.
obtenerImagenHabitacion_idExistente_retornaBytes	Imagen de habitación insertada es recuperable.
obtenerImagenHabitacion_idInexistente_retornaNull	ID inexistente retorna null.
eliminarImagenHotel_eliminalImagen_noObtenible	Eliminar imagen de hotel la remueve de Oracle.
eliminarImagenHabitacion_eliminalImagen_noObtenible	Eliminar imagen de habitación la remueve de Oracle.
obtenerImagenAmenidad_idInexistente_retornaNull	Amenidad inexistente retorna null.

3.5 SERVICES — Unitarios (30 suites · 282 tests)

(Los 17 servicios del informe original se mantienen con sus tests completos.)

Servicios NUEVOS agregados:

AerolineaAdminServiceTest [7 tests]

```
@Test
void listarTodas_conAerolineas_retornaListaCompleta() {
    when(repository.listarTodas()).thenReturn(List.of(
        Map.of("id", 1, "nombre", "Avianca"),
        Map.of("id", 2, "nombre", "Copa Airlines")
    ));
    List<Map<String, Object>> resultado = service.listarTodas();
    assertEquals(2, resultado.size());
    verify(repository).listarTodas();
}
```

Nombre del test	Qué verifica
listarTodas_conAerolineas_retornaListaCompleta	Listar aerolíneas retorna lista del repositorio.
listarTodas_sinAerolineas_retornaListaVacía	Sin aerolíneas retorna lista vacía sin error.
crear_datosValidos_retornaMapaConId	Crear aerolínea con datos válidos retorna mapa con ID.
crear_nombreNulo_lanzaIllegalArgumentException	Nombre null es rechazado antes de llamar al repositorio.
editar_exitoso_delegaAlRepositorio	Editar aerolínea delega correctamente al repositorio.
editar_noExiste_lanzaIllegalArgumentException	Editar aerolínea inexistente lanza error.
listarWebserviceLibres_retornaLista	Listar usuarios webservice disponibles retorna lista.

AerolineaWebserviceServiceTest [7 tests]

Nombre del test	Qué verifica
listarPorUsuario_exitoso_retornaLista	Listar aerolíneas del usuario retorna lista.

listarPorUsuario_sinAerolineas_retornaVacía	Sin aerolíneas del usuario retorna lista vacía.
crear_datosValidos_retornaMapaConId	Crear aerolínea webservice retorna ID.
crear_errorValidacion_lanzaException	Datos inválidos al crear lanzan excepción.
cambiarEstado_activo_delegaAlRepositorio	Cambiar a activo delega al repositorio.
cambiarEstado_cerrado_delegaAlRepositorio	Cambiar a cerrado delega al repositorio.
editar_exitoso_delegaAlRepositorio	Editar delega correctamente al repositorio.

AgenciaNotificadorExternoServiceTest [NUEVO — 6 tests]

Desafío: El constructor original creaba HttpClient internamente (imposible mockear). Se agregó constructor package-private para inyección:

```
// Agregado al código fuente (package-private):
AgenciaNotificadorExternoService(HttpClient httpClient, ObjectMapper mapper) {
    this.httpClient = httpClient;
    this.mapper = mapper;
}
```

Nombre del test	Qué verifica
notificarCancelacion_http200_retornaEnviadoTrue	HTTP 200 del servidor externo → enviado=true.
notificarCancelacion_http500_retornaEnviadoFalse	HTTP 500 → enviado=false con el status code.
notificarCancelacion_timeout_retornaEnviadoFalse	Timeout de red → enviado=false sin propagar excepción.
notificarCancelacion_sinUrl_retornaEnviadoFalse	Agencia sin URL configurada → enviado=false.
notificarCancelacion_sinToken_retornaEnviadoFalse	Agencia sin token → enviado=false.
notificarCancelacion_noEsAgencia_retornaEnviadoFalse	Reservación de usuario directo → enviado=false.

HandshakeAerolineaServiceTest [3 tests]

Nombre del test	Qué verifica
procesarHandshake_urlValida_retornaResponseDTO	URL registrada genera tokens y retorna DTO de respuesta.
procesarHandshake_urlNoRegistrada_lanzaException	URL no registrada lanza excepción antes de guardar tokens.

procesarHandshake_tokensNoGuardados_lanzaException	Si el repositorio no puede guardar, el handshake falla.
--	---

TokenAerolineaServiceTest [5 tests]

Nombre del test	Qué verifica
generarToken_tokenValido_retornaDto	Token válido genera descuento aplicado y retorna el DTO.
generarToken_ciudadNoEncontrada_lanzaException	Ciudad inexistente en Oracle lanza excepción.
generarToken_paisInvalido_lanzaException	País que no coincide con la ciudad lanza excepción.
generarToken_tokenInvalido_lanzaException	Token de aerolínea no reconocido lanza excepción.
generarToken_descuentoCero_aplicaCero	Descuento de 0% aplica pero no modifica el precio.

TokenValidacionServiceTest [4 tests]

Nombre del test	Qué verifica
validar_tokenValido_retornaDtoValido	Token válido y no usado retorna DTO con valido=true.
validar_tokenInvalido_retornaDtoInvalido	Token no registrado retorna valido=false.
validar_tokenExpirado_retornaDtoInvalido	Token expirado retorna valido=false con razón.
validar_tokenYaUsado_retornaDtoInvalido	Token ya marcado como usado retorna valido=false.

BusquedaAerolineaServiceTest [6 tests]

Nombre del test	Qué verifica
buscar_tokenInvalido_lanzaException	Token de aerolínea desconocido lanza excepción.
buscar_ciudadNoEncontrada_lanzaException	Ciudad inexistente lanza excepción con mensaje descriptivo.
buscar_parametrosValidos_retornaLista	Búsqueda exitosa retorna lista de hoteles.
buscar_parametrosValidos_guardaBusqueda	Búsqueda válida registra el evento en el repositorio.
buscar_conDescuento_aplicaDescuentoAlPrecio	El descuento de la aerolínea se aplica a los precios.
buscar_sinDescuento_retornaPrecioNormal	Aerolínea con 0% no modifica el precio.

Los servicios del informe original (AdminReservacionService, AgenciaService, AuthService, BusquedaAgenciaService, BusquedaService, CancelacionService, ComentarioService, DestinosService, DownsService, EmailReservacionService, ExpiracionService, HandshakeService, HotelAgenciaService, HotelService, ImagenService, PagoAgenciaService, PagoService, PdfReservacionService, ReservacionAgenciaService, ReservacionService, SesionService, UsuarioService) se mantienen con todos sus tests — ver Sección 3.6 del informe original.

3.6 SERVICES — Integración contra Oracle (6 suites · 22 tests)

(Existentes desde el informe original)

AdminBusquedaServiceIntegrationTest [5 tests]

Patrón de referencia del proyecto. Inserta usuario y búsquedas propias en Oracle, verifica todos los filtros del dashboard con datos reales.

AuthServiceIntegrationTest [4 tests]

Login exitoso con username, login exitoso con correo, falla usuario inexistente, falla contraseña incorrecta — todo contra Oracle real.

BusquedaServiceIntegrationTest [4 tests]

Búsqueda con ciudad real, registro en tabla Busqueda, búsqueda anónima con UsuarioID null, falla con ciudad inexistente.

CancelacionServiceIntegrationTest [3 tests]

Cancelación exitosa persistida en Oracle, falla con usuario incorrecto, falla con check-in en menos de 24 horas.

PagoServiceIntegrationTest [3 tests]

Pago exitoso que confirma reservación y crea factura, falla reservación de otro usuario, falla CVV inválido.

ReservacionServiceIntegrationTest [3 tests]

Crear reservación real en Oracle, falla sin habitaciones, falla con fecha pasada.

3.7 DTOs (1 suite · 79 tests)

DtosTest [62 clases · 79 tests]

Los DTOs son **POJOs** (Plain Old Java Objects) sin lógica de negocio. Los tests validan que cada clase puede instanciarse, que los setters almacenan el valor asignado, que los getters retornan ese mismo valor y que los constructores paramétricos inicializan correctamente todos los campos.

Estrategia aplicada:

- DTOs con setters (55 clases): prueba de round-trip setter/getter y valores por defecto
- DTOs con constructor paramétrico sin setters (4 clases): prueba que el constructor almacena cada argumento
- DTOs con constructor + setters (3 clases): prueba constructor y sobrescritura vía setter

Clases con constructor paramétrico:

DTO	Constructor	Parámetros
LoginResponseDTO	(String mensaje, String username, int rolId)	mensaje, username, rolId
PuedeCancelarDTO	(boolean puedeCancelar, String razon)	puedeCancelar, razon
TokenAerolineaResponseDTO	(String token, String urlRedireccion, String fechaExpiracion)	token, urlRedireccion, fechaExpiracion
UsuarioValidacionResponseDTO	(boolean usernameExiste, boolean correoExiste, boolean pasaporteExiste)	tres booleans
TokenValidacionResponseDTO	(String ciudad, String pais, double porcentajeDescuento, String fechaExpiracion)	ciudad, pais, descuento, expiración
AerolineaIdentidadDTO	(int id, String nombre, String urlAerolinea)	id, nombre, url

AmenidadDTO	(int id, String nombre) + setters	id, nombre
PaisDTO	(int id, String nombre) + setters	id, nombre
HandshakeResponseDTO	(String tokenSalida) + setter	tokenSalida

Ejemplo de test — DTO con setters:

```
@Test
@DisplayName("BusquedaRequestDTO - setters y getters funcionan")
void busquedaRequestDTO_settersYGetters_funcionan() {
    BusquedaRequestDTO dto = new BusquedaRequestDTO();
    dto.setPais("Guatemala");
    dto.setCiudad("Ciudad de Guatemala");
    dto.setFechaCheckIn("2026-05-01");
    dto.setFechaCheckOut("2026-05-05");
    dto.setCantidadPersonas(2);

    assertEquals("Guatemala",          dto.getPais());
    assertEquals("Ciudad de Guatemala", dto.getCiudad());
    assertEquals("2026-05-01",          dto.getFechaCheckIn());
    assertEquals("2026-05-05",          dto.getFechaCheckOut());
    assertEquals(2,                     dto.getCantidadPersonas());
}
```

Ejemplo de test — DTO con constructor:

```
@Test
@DisplayName("PuedeCancelarDTO - constructor false almacena valores correctos")
void puedeCancelarDTO_constructorFalse_almacenaCorrecto() {
    PuedeCancelarDTO dto = new PuedeCancelarDTO(false, "Ya superó el plazo");
    assertFalse(dto.isPuedeCancelar());
    assertEquals("Ya superó el plazo", dto.getRazon());
}
```

Casos especiales cubiertos:

- Campos Integer nullable (comentarioPadreId, resena, ciudadId): se verifica que aceptan null sin excepción
- Listas (List<HabitacionReservaRequestDTO>): se asignan vacías y con un elemento
- Campos double: comparados con delta 0.001
- Campos boolean: incluyendo isAuthenticated() con valor por defecto false

3.8 MODELS (1 suite · 7 tests)

UsuarioModelTest [Modelo Usuario · 7 tests]

Nombre del test	Qué verifica
constructorVacio_creaInstanciaNoNula	El constructor vacío de Java crea un objeto no nulo.
settersYGetters_camposTexto_funcionan	username, correo, contrasena, pasaporte, nombre, apellido y telefono se almacenan y recuperan correctamente.
settersYGetters_camposNumericos_funcionan	id, rolId y ciudadId (int) se almacenan y recuperan correctamente.
setterYGetter_fechaNacimiento_funciona	Se asigna un LocalDate, se recupera y se verifican año, mes y día individualmente.
ciudadId_aceptaNull	setCiudadId(null) no lanza excepción y getCiudadId() retorna null (campo opcional).
valoresPorDefecto_sonNullOCero	Recién construido, todos los campos String son null, id=0 y rolId=0.
ciudadId_conValor_seAlmacenaCorrectamente	Integer.valueOf(10) se asigna y se recupera sin pérdida de tipo.

Ejemplo de test:

```
@Test
@DisplayName("ciudadId acepta null (campo opcional)")
void ciudadId_aceptaNull() {
    Usuario usuario = new Usuario();
    usuario.setCiudadId(null);
    assertNull(usuario.getCiudadId());
}
```

```
}
```

3.9 DATA (2 suites · 11 tests)

DataAccessExceptionTest [Excepción personalizada · 6 tests]

DataAccessException envuelve excepciones de base de datos en una RuntimeException para que los repositorios no expongan SQLException a las capas superiores.

Nombre del test	Qué verifica
constructor_conMensajeYCausa_almacenaAmbosValores	getMessage() y getCause() retornan los valores pasados al constructor.
esSubclase_deRuntimeException	assertInstanceOf(RuntimeException.class, ex) pasa.
sePuedeLanzarYCapturar_comoRuntimeException	Un bloque assertThrows(RuntimeException.class, ...) captura la excepción.
sePuedeCapturar_comoDataAccessException	También se puede capturar de forma específica.
causa_puedeSerSQLException_simulada	Se pasa un java.sql.SQLException como causa y se recupera correctamente.
mensajeDescriptivo_sePropaga_correctamente	El mensaje puede contener nombres de repositorios y métodos para diagnóstico.

Ejemplo de test:

```
@Test
@DisplayName("causa puede ser SQLException simulada")
void causa_puedeSerSQLException_simulada() {
    Exception sqlEx = new java.sql.SQLException("Constraint violation");
    DataAccessException ex = new DataAccessException("Fallo al insertar", sqlEx);

    assertNotNull(ex.getCause());
    assertEquals("Constraint violation", ex.getCause().getMessage());
}
```

ResultSetMapperTest [Interfaz funcional · 5 tests]

ResultSetMapper<T> es una interfaz funcional con un único método map(ResultSet rs) throws SQLException. Su rol es convertir una fila de Oracle en un objeto Java dentro de DatabaseManager.executeQuery().

Nombre del test	Qué verifica
implementacion_conLambda_mapeaStringCorrectamente	Una lambda que llama rs.getString("nombre") retorna el valor esperado.
implementacion_conLambda_mapeaEnteroCorrectamente	Una lambda que llama rs.getInt("id") retorna el entero correcto.
implementacion_conLambda_mapeaDoubleCorrectamente	Una lambda que llama rs.getDouble("precio") retorna el double correcto.
implementacion_conLambda_construyeObjetoCompuesto	Una lambda puede construir un Object[] con tres columnas distintas.
implementacion_propagaSQLException_cuandoResultSetFalla	Si el mock lanza SQLException, la interfaz la propaga sin envolverla.

Ejemplo de test:

```
@Test
@DisplayName("implementacion con lambda construye objeto compuesto")
void implementacion_conLambda_construyeObjetoCompuesto() throws SQLException {
    ResultSet rs = mock(ResultSet.class);
    when(rs.getInt("id")).thenReturn(1);
    when(rs.getString("nombre")).thenReturn("Miku Inn");
    when(rs.getDouble("rating")).thenReturn(4.8);

    ResultSetMapper<Object[]> mapper = resultSet -> new Object[]{
        resultSet.getInt("id"),
        resultSet.getString("nombre"),
        resultSet.getDouble("rating")
    };

    Object[] resultado = mapper.map(rs);
    assertEquals(1, resultado[0]);
    assertEquals("Miku Inn", resultado[1]);
    assertEquals(4.8, resultado[2]);
}
```

```
}
```

Nota: DatabaseManager no tiene tests directos porque ya queda exhaustivamente ejercido por los **115 tests de integración** de los 19 repositorios (~300+ invocaciones a executeQuery, executeUpdate y executeInsertReturnId).

3.10 CONFIG (1 suite · 2 tests)

ServerConfigTest [Smoke tests · 2 tests]

ServerConfig levanta el servidor Javalin con CORS, puerto configurable y lista de orígenes adicionales. No se levanta el servidor real en los tests (sería una prueba E2E que requeriría puerto libre y teardown).

Nombre del test	Qué verifica
claseServerConfig_existe_esPublica	ServerConfig.class existe y tiene modificador public.
metodoCreateServer_existe_esPublicoEstatico	El método createServer() existe, es public y es static.

Ejemplo de test:

```
@Test
@DisplayName("metodo createServer existe y es publico estatico")
void metodoCreateServer_existe_esPublicoEstatico() throws NoSuchMethodException {
    Method m = ServerConfig.class.getDeclaredMethod("createServer");
    assertTrue(Modifier.isPublic(m.getModifiers()));
    assertTrue(Modifier.isStatic(m.getModifiers()));
}
```

Justificación del alcance: Los controladores y servicios (los 553 tests de esas categorías) ya validan que la lógica de negocio funciona. Lo que importa de ServerConfig es que el método de arranque es invocable; el comportamiento real se verifica a través de los tests de integración que conectan a Oracle.

4. Correcciones Técnicas Aplicadas

4.1 AdminReservacionServiceTest — verificaciones faltantes

Problema: El método cancelarReservacion() retorna ResultadoNotificacionDTO pero los tests originales no lo capturaban ni validaban.

```
// ANTES (incompleto):
service.cancelarReservacion(10, "motivo");
verify(repo).cancelarReservacion(10, "motivo");

// DESPUÉS (completo):
ResultadoNotificacionDTO resultado = service.cancelarReservacion(10, "motivo");
assertNotNull(resultado);
verify(repo).cancelarReservacion(10, "motivo");
verify(notificadorAgencia).notificarCancelacion(10, "motivo");
```

4.2 Chaining de Context en Controllers (9 archivos)

Problema: ctx.status(200).json(service.method()) registra el status 200 antes de evaluar el servicio. Si el servicio lanza una excepción, el 200 ya fue marcado.

```
// INCORRECTO:
ctx.status(200).json(service.procesarPago(id, dto));

// CORRECTO:
PagoResponseDTO resultado = service.procesarPago(id, dto);
ctx.status(200);
ctx.json(resultado);
```

Archivos corregidos: PagoController, ReservacionController, AgenciaController, ComentarioController, TokenAerolineaController, TokenValidacionController, PagoAgenciaController, ReservacionAgenciaController.

4.3 Pre-limpieza defensiva en tests de integración

Problema: Los tests de integración que corren múltiples ciclos (uno por @Test) podían dejar residuos en Oracle si un @AfterEach fallaba silenciosamente.

Solución: Agregar bloque de limpieza al inicio de @BeforeEach usando identificadores fijos:

```
@BeforeEach
void setUp() {
    // 0. Pre-limpieza defensiva
    DatabaseManager.executeUpdate(
        "DELETE FROM DetallesReservacion WHERE ReservacionID IN " +
        "(SELECT ID FROM Reservacion WHERE No_Reservacion = ?)", NO_RESERVACION);
    DatabaseManager.executeUpdate(
        "DELETE FROM Reservacion WHERE No_Reservacion = ?", NO_RESERVACION);
    DatabaseManager.executeUpdate(
        "DELETE FROM Agencia WHERE URL_Agencia = ?", "http://it-res-agencia.com");
    DatabaseManager.executeUpdate(
        "DELETE FROM Usuario WHERE Username = ?", USERNAME);
    // ... continúa con inserciones normales
}
```

5. Pruebas Frontend (Svelte + Jest)

5.1 Resumen Frontend

Métrica	Valor
Total tests	90
Tests pasaron	90
Tests fallaron	0
Suites	2

Framework	Jest 30.3.0 + Babel
Entorno	jsdom

5.2 registerUtils.test.js [90 tests]

Prueba las 6 funciones puras del módulo src/lib/registerUtils.js:

validatePassword [~18 tests]

```
describe('validatePassword', () => {
  test('acepta contraseña válida', () => {
    const result = validatePassword('MiPass123!');
    expect(result.minLength).toBe(true);
    expect(result.hasUpperCase).toBe(true);
    expect(result.hasLowerCase).toBe(true);
    expect(result.hasNumber).toBe(true);
    expect(result.hasSpecial).toBe(true);
  });
  test('rechaza sin mayúscula', () => {
    expect(validatePassword('mipass123!').hasUpperCase).toBe(false);
  });
});
```

Grupo	Tests	Qué verifica
minLength	3	Mínimo 8 caracteres: 7, exactamente 8, más de 8
hasUpperCase	3	Con/sin/solo mayúsculas
hasLowerCase	3	Con/sin/solo minúsculas
hasNumber	3	Con/sin/solo números
hasSpecial	3	Con/sin/solo especiales
Combinaciones	3	Vacía, todos los requisitos, ninguno

validateForm — TC-70: corrección hasSpecial [~38 tests]

Corrección aplicada: validateForm no validaba hasSpecial. La condición estaba incompleta:

```
// ANTES (incompleto – no validaba hasSpecial):
else if (!passwordValidation.minLength || !passwordValidation.hasUpperCase ||
         !passwordValidation.hasLowerCase || !passwordValidation.hasNumber)
  errors.password = 'La contraseña no cumple los requisitos';

// DESPUÉS (completo – TC-70):
else if (!passwordValidation.minLength || !passwordValidation.hasUpperCase ||
         !passwordValidation.hasLowerCase || !passwordValidation.hasNumber ||
         !passwordValidation.hasSpecial) // ← agregado
  errors.password = 'La contraseña no cumple los requisitos';
```

TC-70 agregado al test:

```
test('TC-70 | contraseña sin carácter especial → "La contraseña no cumple los requisitos"', () => {
  const s = estadoValido();
  s.formData.password = 'Abcdefg1'; // sin especial
  s.formData.confirmPassword = 'Abcdefg1';
  s.passwordValidation = validatePassword('Abcdefg1');
  expect(validateForm(s).password).toBe('La contraseña no cumple los requisitos');
});
```

Otras funciones

Función	Tests	Qué verifica
getPasswordStrength	~8	Niveles: vacía, débil, regular, fuerte, muy fuerte
calculateAge	~8	Menor de edad, mayor, exactamente 18, fix de timezone
formatLocalPhone	~12	GT (8 dígitos), MX (10), US (10), vacío, otro país
getPhonePlaceholder	~6	GT, MX, US, otro país, sin país seleccionado

5.3 Correcciones Frontend

1. **Import de registerUtils.js en Register.svelte:** Se eliminaron las 6 funciones definidas inline (~100 líneas) y se importan desde el módulo compartido.
 2. **Test muerto eliminado:** src/utils/validarFechas.test.js fue eliminado (nunca descubierto por Jest).
 3. **Dependencia jsdom:** Se instaló jest-environment-jsdom y @testing-library/jest-dom para permitir tests con DOM.
-

6. Integridad del Código

6.1 Sin modificaciones al código de producción (salvo excepciones documentadas)

La única modificación al código fuente fue en AgenciaNotificadorExternoService: se agregó un constructor package-private para permitir inyección de HttpClient en los tests. Esta modificación es **no invasiva** (no cambia el comportamiento en producción).

6.2 Garantías de calidad

- **Sin valores hardcodedos forzados:** Los resultados esperados surgen de la lógica real, no de valores inventados.
- **Sin catch que tragan errores:** No existen bloques try-catch que ignoren excepciones para hacer pasar tests artificialmente.
- **Sin Strictness.LENIENT:** Cada stub configurado es efectivamente invocado.
- **Cobertura de caminos de error:** Además del flujo normal, cada servicio tiene tests de datos inválidos y casos de borde.
- **Integración real:** Los 115 tests de repositorios y 22 tests de servicios usan Oracle real, no H2 ni mocks de BD.

6.3 Casos técnicos especiales

- **BCrypt real:** AuthService y UsuarioService usan PasswordHelper.hash() real, sin MockedStatic.
 - **Reflexión para método privado:** ExpiracionServiceTest usa Java Reflection para invocar expirarReservaciones() sin modificar producción.
 - **Fechas futuras:** Tests de reservaciones usan fechas en 2027-2040 para superar la validación de check-in futuro.
 - **Pre-limpieza defensiva:** Tests de integración limpian residuos de ciclos anteriores usando identificadores fijos.
-

7. Conclusión

Las **912 pruebas** del sistema Miku Inn (822 backend + 90 frontend) pasaron satisfactoriamente, cubriendo los 89 componentes del microservicio Hotel y las utilidades del frontend Svelte.

La expansión respecto al informe original (249 → 822 tests en backend) se logró mediante:

1. **Controllers:** 28 suites verificando cada endpoint con sus respuestas HTTP correctas
2. **Repositories:** 19 suites de integración real contra Oracle que validan SQL, FK y persistencia
3. **Services nuevos:** 7 servicios del módulo aerolíneas/tokens/notificador externo
4. **Middleware:** 2 nuevas suites para autenticación de agencias y aerolíneas
5. **DTOs/Models/Data/Config:** 5 suites adicionales que completan la cobertura al 100% de todos los componentes del proyecto

El proyecto está en condiciones de ser entregado con confianza en la calidad del código probado.

Miku Inn · Sistema de Gestión Hotelera · Guatemala City, Guatemala

Comandos de Verificación

```
# Backend – ejecutar todos los tests
cd "C:\Proyecto-Final-ADFS-y-DABD1\Cadena de Hoteles\Hotel"
mvn clean test
# → Tests run: 822, Failures: 0, Errors: 0  ✓  BUILD SUCCESS
```

```
# Frontend – ejecutar todos los tests
cd "C:\Proyecto-Final-ADFS-y-DABD1\Cadena de Hoteles\Miku Inn"
npm test
# → Tests: 90 passed, 90 total  ✓
```

```
# Backend – regenerar documentación Tests.md
.\generar-tests-doc.ps1
```